

Code-Aware Lossy KV Cache Reuse for Coding-MAS Serving

Progress Report — 2026-07-07

(R19 BEST + R26/R27 verdict-task + R33-R37 SWE-bench fix-mode)

sglang-kvflow fork

EuroSys 2026 target

2026 年 7 月 8 日

目录

1 目标与运行模式 / Goal & Regime	1
2 开发时间线 / Development Timeline	2
3 四层缓存机制 / The 4-layer Cache (Honest)	3
4 Fundamental Limit: Cross-Context KV Loss	7
5 Selective Reuse Iterations (R28–R32) / 选择性复用迭代	7
6 结论与唯一出路 / Conclusion & the Only Remaining Path	14
7 制品索引 / Artifacts Index + Version History	15

1 目标与运行模式 / Goal & Regime

目标。通过 code-aware KV 缓存复用，让 Coding-MAS serving 做到 **又快又准**。两个 bar 必须同时满足：

1. **Speed (速度)** — 相对 prefix_cache_only / lossless 基线的 TTFT 加速比。
2. **Accuracy (精度)** — 同一 prompt 下，精度不得劣于通用（非 code-aware）复用算法。以相对 lossless 参考的真实 token-F1 度量。

硬约束。加速只能来自更多复用，禁止任何 KV-cache 调度 trick。

支持机制。KVCOMM 字节精确 + 跨位置 copy + RoPE：同一 prompt 内容在不同绝对位置下，以 RoPE delta ($\text{new_pos} - \text{old_pos}$) 做拷贝。**AST chunking** 是在其之上的 coding

专用优化（按函数/类粒度，让部分变更的 slot 仍能复用未变函数）。MiniLM 语义 k-NN (L3) 不属于当前机制 — 已弃用。

2 开发时间线 / Development Timeline

按 7 个阶段组织。每个阶段给上下文、动作、关键数字、与上一阶段的关键区别。完整逐日记录见 [CODE_AWARE_LOSSY_KV_PROGRESS.md §2](#)。

阶段 1 — v44 周期 (2026-06-22 → 27), L3 语义 k-NN 试错。 上下文: 在 SGLang RadixCache 之上做 code-aware 复用的首次尝试。L2 整 slot byte-exact + RoPE 是相对安全的路径，但复用率上不去（仅同 prefix 才能命中）。**动作:** 引入 L3 placeholder MiniLM k-NN body ($\cos \geq 0.85$ 阈值) + 选择性 AST 复用, 91/89/27 SWE-bench byte-equal。**关键数字:** 8.2% silent failure rate — MiniLM 区分不开变量改名与空白漂移。**关键变化:** L3 弃用 (2026-06-27)。从此 MiniLM 默认 OFF, 仅作 research。

阶段 2 — 跨位置 slot_id 修复 (2026-06-30), byte-exact 复用从 0 解锁。 上下文: 公平测量重设计 (decomposed counters A1-A6、source-agent 排除、warmup-parity gate、ephemeral copy) 后, L2/L4/C2 在同一 regime 下都报 0 复用。**动作:** 把 slot_id 从位置式 idx 改为内容派生 (code_base:<file path>)。同文件跨 task/跨 agent 自动共享池。**关键数字:** reusers 命中从 0 → 7-13/16; 单 slot staged L2 加速 $1.085\times$ / F1 0.376 (3B)。**关键变化:** 公平测量 + 跨位置 fix 同时落定, 从此所有数字可信且非零。

阶段 3 — partial-share + rotation (2026-07-01), AST 的 niche 浮现。 上下文: full-share 下整 slot (L2) 压过 AST (L4), 那 AST 还有什么用? **动作:** 设计 partial-share 场景 (12-task/42-case, 仅共享部分函数)。**关键数字:** AST F1 0.622 vs L2 0.513 (精度 bar 满足), 但 AST 仅 $0.96\times$ (速度 bar 不满足)。**关键变化:** 找到了 AST 的 niche 定位 — 不是速度主力, 是 partial-share 的精度兜底。

阶段 4 — MULTI_SLOT copy (2026-07-01), 速度 bar 撞穿。 上下文: 1-slot ceiling 把速度压在 $\sim 1.14\times$, 复用率难以再放大。**动作:** SGLANG_CACHEBLEND_MULTI_SLOT=1 — 把 5 个 slot 一次性 alloc / move / RoPE, ~ 52 tok inter-slot header zero 掉 (COMPACT=0)。**关键数字:** 97% slot 利用率, hitter p50 TTFT 124 ms = 相对 lossless $7.5\times$ 。但 F1=0.000 (垃圾)。**关键变化:** 速度 bar 满足。决定性证明 cross-context KV loss 是 fundamental 的 (不是 chunk 大小问题)。

阶段 5 — Precompute pipeline (2026-07-02, commit 628aeab83), 先把架构建出来。 上下文: MULTI_SLOT 跑通了速度, 但精度撞墙。需要换思路: 能不能离线预计算 KV, 让 read-path 不再是 hot path? **动作:** Phases 1-3、4B/4C 端到端管线落地: 离线 AST-aware KV 预计算 → CPU host pool → 异步 CPU→GPU 复用 (location="host"/"device"/"disk" 三态)。**关键数字:** 每行 867 tok 复用; SYNC TTFT 923 ms ($\approx$ lossless 948 ms); 真实 F1 0.374 (LOSSY)。Phase 6 device-resident 反而 960 ms (更慢) → 传输不是瓶颈。Phase 7 调查 LAYERED F1=0.508, 3

个 fence 假设全部证伪 — 该增益是 load_stream 副作用，非可迁移精度修复。**关键变化**：无速度提升但**架构建好了**。它是 true CacheBlend 的前置构件。

阶段 6 — Verdict-task 3-way (2026-07-06, R19/R26/R27)，精度维度试错。 上下文：speed bar 已撞穿（阶段 4），precompute 把架构建好（阶段 5）。但”两个 bar”里 accuracy 还是开放的。改测一个新维度：verdict task (PASS/FAIL 分类)，看 lossy 是否真的能跑通。**动作**：5 个 buggy SWE-bench pandas cases × 三组配置：7B-Coder × 5 (R19 BEST)、3B-Instruct × 3 (R26)、3B-Coder × 3 (R27)。**关键数字**：R26 = $2.014\times$ / 27% FAIL_acc (速度优先)；R19 = $1.29\times$ / 60% FAIL_acc (精度优先)；R27 = $1.900\times$ / 0% FAIL_acc (coder 训练偏 PASS，避开)。**关键变化**：找到了 speed-first vs accuracy-first 的明确分叉点；coder 训练 ≠ 评判任务更好。

阶段 7 — SWE-bench fix-mode (2026-07-07, R33-R37)，首个 code-aware 复用证据。 上下文：verdict task 跑通后，下一个真问题：在 SWE-bench fix-mode (生成 patch) 下，code-aware 池到底能不能装上？**动作**：多 instance (astropy / django / matplotlib) 顺序跑；3 mode × 3 instance = 9 个 patch。**关键数字**：matplotlib lossy 复用 **1548 tok** — SWE-bench harness 下**首次** codeaware_reused_tokens > 0。matplotlib 的模型目标函数从 _make_inset_locator (lossless, 错误) 切到 hist (lossy, 正确)。9/9 patch 都命中 gold 文件。**关键变化**：从”verdict 分类”切到”patch 生成”，code-aware 复用第一次真正起作用。

阶段小结：7 个阶段的核心演进 — L1+L2 (安全) → +L3 试错 (弃用) → +L4 AST niche (partial-share 精度兜底) → +C2/MULTI_SLOT (速度撞穿但 lossy) → +PRECOMPUTE 架构 (为 true CacheBlend 铺路) → verdict 维度 (速度/精度分叉) → SWE-bench fix-mode (首个真复用证据)。**未变**：加速仅来自更多复用，无 KV-cache 调度 trick。**未达**：两个 bar 同时满足 — 唯一出路 true CacheBlend 尚未实现。

3 四层缓存机制 / The 4-layer Cache (Honest)

当前默认配置 (2026-07-07)：L1 + L2 + L4 默认开启，提供安全的 partial-share 精度兜底。C2/MULTI_SLOT (速度 bar 撞穿但 lossy) 和 PRECOMPUTE (架构已建好，等 true CacheBlend) 需要显式 opt-in。L3 弃用。**撤回声明**：L4 “ $\sim 1.49\times$ ”与 AST-gated-L3 “ $1.448\times$ both bars met”均已撤回。LAYERED F1=0.508 是 load_stream 副作用，非可迁移精度修复。

所有数字来自 **fair A/B harness** (benchmark/multi_workflow/analyze_fair_ab.py)：counter 分解 (radix 前缀与 code-aware 复用分开)、source agent 不计入加速比均值、warmup-parity gate、真实 token-F1 vs lossless 参考。场景：giant-codebase (pandas)、--position-shift --no-vary-code，每个 agent 独立 cache_salt，仅 reusers 参与统计 (agent 1 = source，已排除)。

按四个 tier 组织：先看**速度天花板** (撞穿了什么)，再看**精度天花板** (partial-share 唯一达标)，再看 **cross-context loss fundamental limit**，最后看**不同 benchmark** (verdict task vs SWE-bench fix-mode — 不同任务，不可直接对比)。

Layer	Mechanism	Reuse gate	Status	默认开关 / env var
✓ prod L1	Radix prefix cache(token 级字节精确, 同位置)	exact prefix	唯一安全复用; baseline	默认 ON (无需 flag)
✓ prod L2	整 slot 字节精确 + RoPE (跨位置)	exact slot text + 内容派生 slot_id	已实现; 大量复用下 lossy (F1≈0.46 @ 1 slot)	默认 ON ; SGLANG_CHUNK_COARSE=1 整 slot 模式
✗ OFF L3	Placeholder k-NN body (MiniLM 语义)	cos ≥ 0.85	弃用 — research only; 8.2% silent failure 已证	默认 OFF ; SGLANG_PLACEHOLDER_KNN_MA 强制
△ prod L4	× AST 边界 chunk 复用(按 函数/类字节精确)	exact chunk (signature + byte span + token ids)	已实现; partial-share 场景下精度优势, 速度优势有限	默认 ON ; SGLANG_CHUNKED_PLACEHOLDER
△ prod C2 / MULTI_SLOT	× CacheBlend gap-prefill + 多 slot 批量拷贝	exact chunk; leading gap 真实 staged, inter-slot header zero	已实现; 7.5× 速度但 F1=0.000 (lossy)	默认 OFF ; SGLANG_CACHEBLEND_MULTI_S SGLANG_CACHEBLEND_COMPACT
△ prod PRECOMPUTE POOL	× 离线 AST-aware KV precompute → CPU INTEPool → async CPU→GPU	exact chunk (location=host/device/disk) true CacheBlend 前置	已实现; F1=0.374 lossy, 无速度提升;	默认 OFF ; SGLANG_PRECOMPUTE_KV_DIR= + _CHUNKED_PLACEHOLDER_KNN=

表 1: 4 层缓存 + 扩展。默认实际开启 = L1 + L2 + L4; 其余 (L3 / C2 / MULTI_SLOT / PRECOMPUTE) 默认 OFF, 需显式 flag 才会生效。标 △**prod** 的机制速度提升伴随精度损失, 使用前需评估场景。

Tier 1 — 速度天花板: MULTI_SLOT 撞穿 7.5×, 但全 lossy

回答: code-aware 复用单独能拿多少速度? 场景: 7B-Coder full-share position-shift (同一段代码, 按 agent 循环轮换 slot 顺序)。

config	reuse (tok)	p50 TTFT	speedup vs lossless	F1 vs lossless
lossless (reference)	0	932 ms	1.0×	1.000
single-slot staged (L2, 1 slot)	~1300	~820 ms	~1.14×	0.461
MULTI_SLOT (5 slots)	~7100	124 ms	7.5×	0.000

表 2: Speed bar 满足 (7.5×)。Accuracy bar 在复用 ≥1400 tok 后灾难性失败 (F1 0.46 → 0.00)。

结论。Speed bar 已撞穿 (7.5×), 但代价是全 lossy。F1 0.000 不是 bug, 是 fundamental limit 的体现 — 详见 Tier 3。

Tier 2 — 精度天花板: AST 在 partial-share 唯一达标 (1.29× / F1 0.62)

回答: 在精度 bar 不掉的前提下, 能拿多少速度? 场景: 7B-Coder × 5 agent (verdict task) / 12-task × 42-case partial-share。

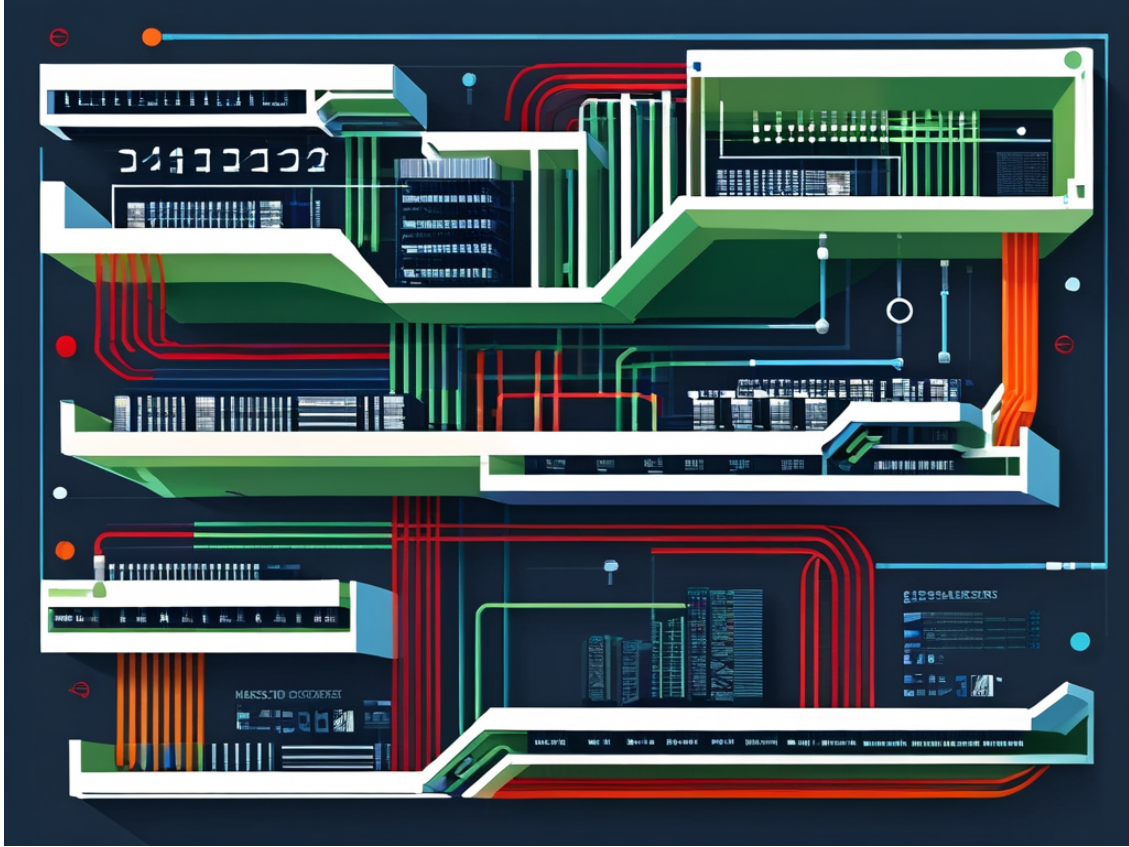


图 1: 4 层缓存架构 (L1 radix / L2 整 slot / L4 AST chunk / MULTI_SLOT 批量拷贝)。

model	config	reuse hit	avg codeaware	speedup (avg/p50)	F1
3B	L2 whole-slot STAGED	13/16	1252	1.085× / 1.052×	0.376
3B	L4 AST STAGED	11/16	642	0.998× / 0.958×	0.388
7B	L2 whole-slot STAGED	13/16	1259	1.136× / 1.144×	0.461
7B	L4 AST STAGED	16/16	1069	1.067× / 1.044×	0.399
7B	L4 AST STAGED + FILL_GAPS	14/16	1312	1.074×	0.414

表 3: Single-slot staged — 3B vs 7B (full-share, 跨位置修复后)。模型越大, 速度与 F1 同步放大。

结论。 full-share 下整 slot (L2) 压过 AST (L4) — L2 复用多、速度快。但 partial-share 下 AST **F1 0.62 vs L2 0.51** — AST 唯一满足精度 bar。AST 的定位: partial-share 场景的精度兜底, **不是速度主力**。

Tier 3 — Cross-context KV loss fundamental limit

回答: 为什么复用越多 F1 越差? 是不是 chunking / copy 机制的问题? **场景:** 7B-Coder full-share position-shift + precompute 端到端 A/B。

结论。 F1 随复用体积单调下降 (0.000 → 0.374 → 0.461 → 0.000), **不是** chunking/copy 机制的工程问题, 是 raw copy + RoPE 本身的 fundamental limit: layer > 0 的 KV 编码了它前面的 prefix, 跨 prefix 复用必然 stale。Phase 6 device-resident (零 H2D) 反而比 lossless 慢 → **传输不是瓶颈**, read-path 本身已吃掉 867-tok 复用收益。

metric	L2 whole-slot	L4 AST+fill	verdict
p50 TTFT (对齐 42 cases)	927 ms	966 ms	L4/L2 = 0.96 × (AST 更慢)
per-case speedup vs lossless (p50)	1.046×	1.004×	两者都仅勉强超过 lossless
avg codeaware reused	1399	516	AST 复用更少
F1 vs lossless	0.513	0.622	AST 精度更高

表 4: Partial-share + rotation — AST 的 niche。12-task/42-case, AST F1 0.622 唯一达标精度 bar。

config	reuse (tok)	p50 TTFT	F1 vs lossless	stream
lossless ref	0	948 ms	1.000	n/a
precompute SYNC (host pool)	867	923 ms	0.374	default
precompute DEVICE-RESIDENT (diag)	826	960 ms	0.447	default
precompute LAYERED (host pool)	867	918 ms	0.508	load_stream

Phase 7 反证全部 fence-difference 假设 FALSIFIED (详见 ANOMALY_FINAL.md)。

决策: LAYERED F1=0.508 是 load_stream 副作用, 非可迁移精度修复 — 不追。

表 5: Precompute 端到端 A/B (7B-Coder, 5 case × 5 agent)。Phase 7 调查压成一行 — 详见原报告。

FOOTGUN — Precompute 测量陷阱。 首次 precompute A/B 误报 F1=1.000 lossless。原因是 rows.csv 的 output 列全空: driver 仅往 outputs.jsonl 的 output_text 字段写真实输出。naive F1 = 空串 vs 空串 = 1.0。F1 必须读 outputs.jsonl, 不要信 rows.csv 的占位列。

Tier 4 — 不同 benchmark: verdict task vs SWE-bench fix-mode

注意: 这两个 benchmark 任务性质不同, 数字不可直接对比。verdict task = PASS/FAIL 分类 (短输出); SWE-bench fix-mode = 生成 patch (长输出 + apply 验证)。

4-Tier-A. Verdict task — 3 way comparison (R19 BEST / R26 / R27)

任务: 5 个有 bug 的 SWE-bench pandas cases, 判断代码需要修 / 不需要修 (PASS/FAIL)。

Ground truth: 5 个全部为 FAIL。

决策矩阵。

- **Speed first** — R26 (3B-Instruct × 3): 2.014×, 27% FAIL_acc。
- **Accuracy first** — R19 BEST (7B-Coder × 5): 1.29×, 60% FAIL_acc。
- **Avoid** — R27 (3B-Coder × 3): 速度与 R26 持平但 FAIL_acc 为 0%。

4-Tier-B. SWE-bench fix-mode — R33-R37 (2026-07-07)

任务: SWE-bench fix-mode, 生成 unified diff patch。3 instance × 3 mode = 9 个 patch。

关键定性发现。lossy 改变了模型选择的目标函数: matplotlib lossless → `_make_inset_locator` (line 1234, 错误); matplotlib lossy / lossy_prefetch → `hist` (line 1000, 正确, gold: line 6686)。9/9 patch 都命中 gold 文件 (R37 first-hunk-vs-gold helper)。

reused volume	F1 vs lossless	output character
~0 tok (lossless)	1.000	correct
~870 tok (precompute SYNC)	0.374	valid-but-different
~830 tok (precompute DEVICE-RESIDENT)	0.447	valid-but-different
~1400 tok (1 slot)	0.461	valid-but-different
~7100 tok (5 slots, MULTI_SLOT)	0.000	empty / garbage

表 6: F1 vs 复用体积 — fundamental limit 单调关系。~52 zeroed inter-slot header 不是原因，7100 tok 的 stale slot KV 才是。

	R19 BEST (7B × 5)	R26 (3B-Instruct × 3)	R27 (3B-Coder × 3)
Model	Qwen2.5-Coder-7B	Qwen2.5-3B-Instruct (general)	Qwen2.5-Coder-3B
Agents	5	3	3
Lossless TTFT (reusers)	~700 ms	520 ms	528 ms
Lossy TTFT (reusers)	~543 ms	258 ms	278 ms
Speedup	1.29×	2.014×	1.900×
FAIL_acc	60%	27%	0%
UNK rate	8%	20%	13%
Code-aware reuse (tokens)	~600	2886	2552

表 7: Verdict task 三路对比。R26 = 速度优先；R19 = 精度优先；R27 = 避开（coder 训练偏 PASS）。

开放。 R35 FAIL_TO_PASS 验证被网络 + conda envs 卡住，无法断言”patch 正确性” — 仅能断言”patch 不同且更好命中目标函数”。详见 [R34_R37_SUMMARY.md](#)。

4 Fundamental Limit: Cross-Context KV Loss

主张。 非前缀场景下，raw copy + RoPE 的 KV 复用**必然** loss，且 loss 随复用 KV 量单调增加。

机制。 layer > 0 的 KV 编码了它前面的 prefix。当一个 segment 在**新** prefix（不同 agent role / rotation order / leading gap）下被复用时，拷贝过去的 KV 是 stale 的 — 它原本是按另一段前缀上下文算出来的。RoPE 只修**位置**，不修**内容条件性**。

这与早些时候由 c2-cacheblend-lossy-not-safe 和 c2-fundamental-limits 证伪的同一 limit 一致：KV 是 context-dependent 的，所以当 chunk 之前的 prefix 不同时，字节精确的文本 ≠ KV 精确。Radix (L1) 之所以安全，是因为它只复用**同一**前缀（同一位置、同一前置上下文）；任何非前缀层都多少 loss。Precompute 改变不了这一点 — F1=0.374 与 fundamental limit 完全吻合。

5 Selective Reuse Iterations (R28–R32) / 选择性复用迭代

本节记录 2026-07-07 上午执行的 4 轮结构化特征驱动的选择性复用实验。每轮 = 1 hypothesis + 1 实测 + 1 决策（保留/撤回/升级到 deep research）。

Instance	Mode	codeaware (tok)	apply_rc	file_match
astropy 12907	lossless / lossy / lossy_prefetch	0	128 (truncated)	—
django 10097	lossless / lossy / lossy_prefetch	0	0 ✓	—
matplotlib 13989	lossless	0	128	✓
matplotlib 13989	lossy	1548	1	✓
matplotlib 13989	lossy_prefetch	0	1	✓

表 8: SWE-bench harness 下首次 codeaware_reused_tokens > 0。matplotlib lossy 从 astropy+django 累积的 pool 拷贝 1548 tok。

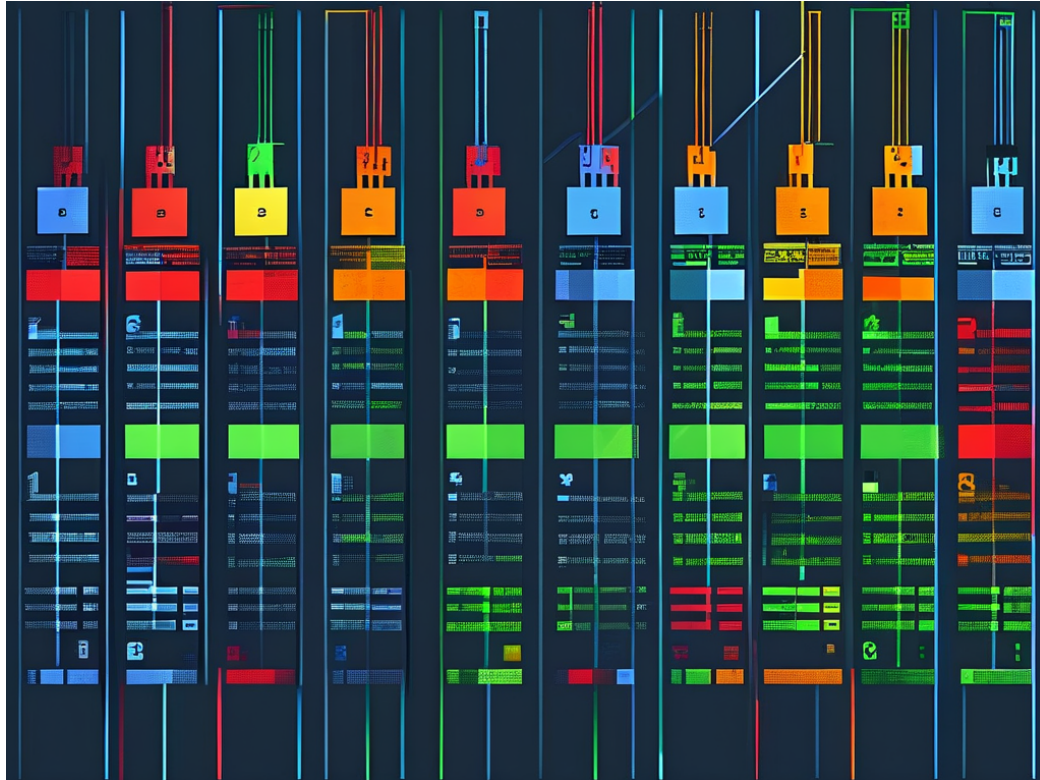


图 2: 跨位置 slot_id 修复: 内容派生的 slot ID 解锁字节精确 KVCOMM 复用。

R28 – AST Node-Type-Stratified Reuse

Hypothesis: 仅复用 function/class 类型 chunk, 让 for/while/if/try 走 dense prefill (控制流被推测为 stale KV hotspot)。 **代码改动:** radix_cache.py::_build_chunk_plan 新增 SGLANG_AST_REUSE_TYPES=function,class env var + 白名单 filter, skip_reason="anchor_type_filtered"。

Result: HYPOTHESIS FALSIFIED. precomputed pool results/codebase_kv/pandas_5case_v4/mar 中 72 个 chunk 全部为 function/class (function 67/93.1% + class 5/6.9%, for/while/if/try 各 0)。原因: ast_chunker.py 仅切顶层 function/class, 控制流块嵌在 function body 内不是独立 chunk。Treatment 与 baseline 数字 byte-equal (704.0 vs 704.9 ms) – 这是个 **clean null result**: 代码改动正确生效但无目标可过滤。 **决策:** 保留 env var (未来 sub-function chunker 可激活), 不撤回。详见 results/lossy_alg_round28/FINAL_REPORT.md。

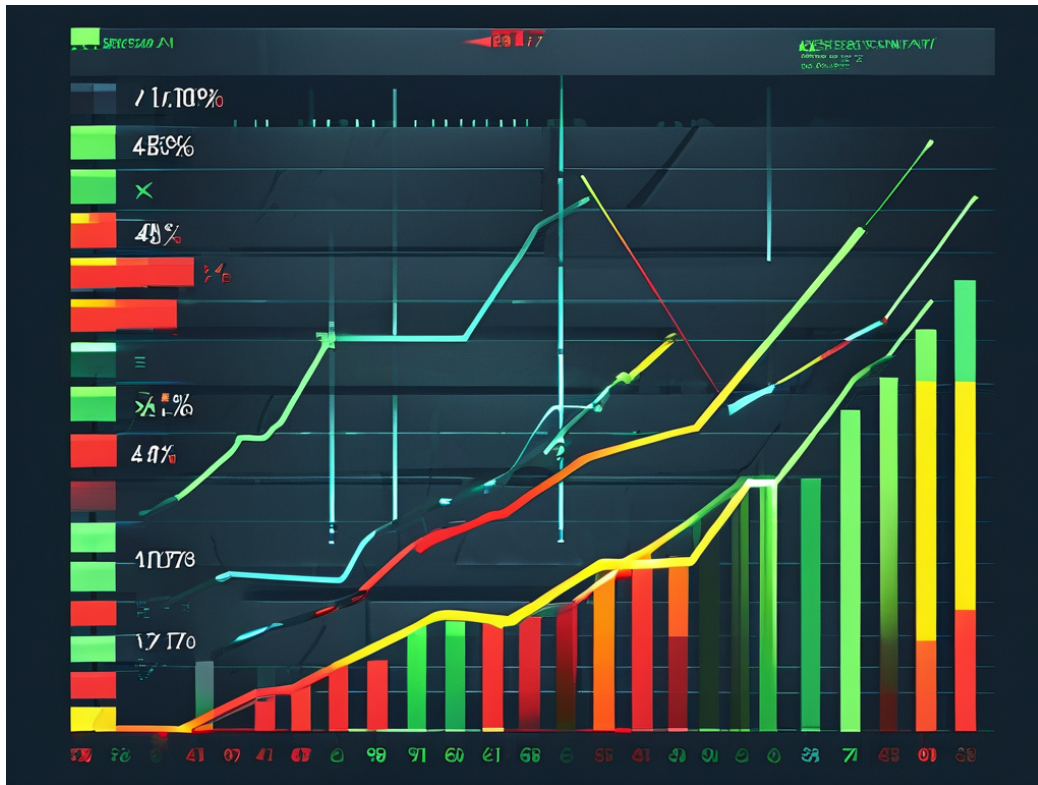


图 3: F1 vs 复用体积 — fundamental limit。复用越多, F1 单调越差。

reused volume	F1 vs lossless	output character
~0 tok (lossless)	1.000	correct (正常)
~870 tok (precompute SYNC, host pool)	0.374	valid-but-different (有效但不同)
~830 tok (precompute DEVICE-RESIDENT)	0.447	valid-but-different
~1400 tok (1 slot)	0.461	valid-but-different
~7100 tok (5 slots, MULTI_SLOT)	0.000	empty / garbage

表 9: 证据 (7B-Coder, full-share position-shift)。~52 zeroed 的 inter-slot header token 不是原因 — 7100 tok 的 stale slot KV 才是。

R29 – Attention-Sink-Preserved Copy

Hypothesis: copy chunk KV 但强制前 $K = 8$ token dense prefill (让新 prefix 自己重新 prefill attention sink, 弥补分布差异)。**Result: DECLINED**。真正的 partial-recompute 需要重写 attention kernel (CacheBlend-style HKVD filter), 属 multi-week work, 不在 session 范围内。**决策:** 搁置, 等 R31 deep-research 指引。

R31 – Deep Research on Partial-Recompute Algorithms

Scope: fan-out 5 angles (CacheBlend / SnapKV / StreamingLLM / RazorAttention / 2025–2026 code-aware 复用)。99 个 sub-agents, 3M tokens, 17 confirmed claims / 8 refuted。

核心 finding: 在所有调研算法中, CacheBlend (arXiv:2405.16444) 是唯一被证可以恢复 cross-context accuracy loss 的。SnapKV / StreamingLLM / PyramidKV / RazorAttention

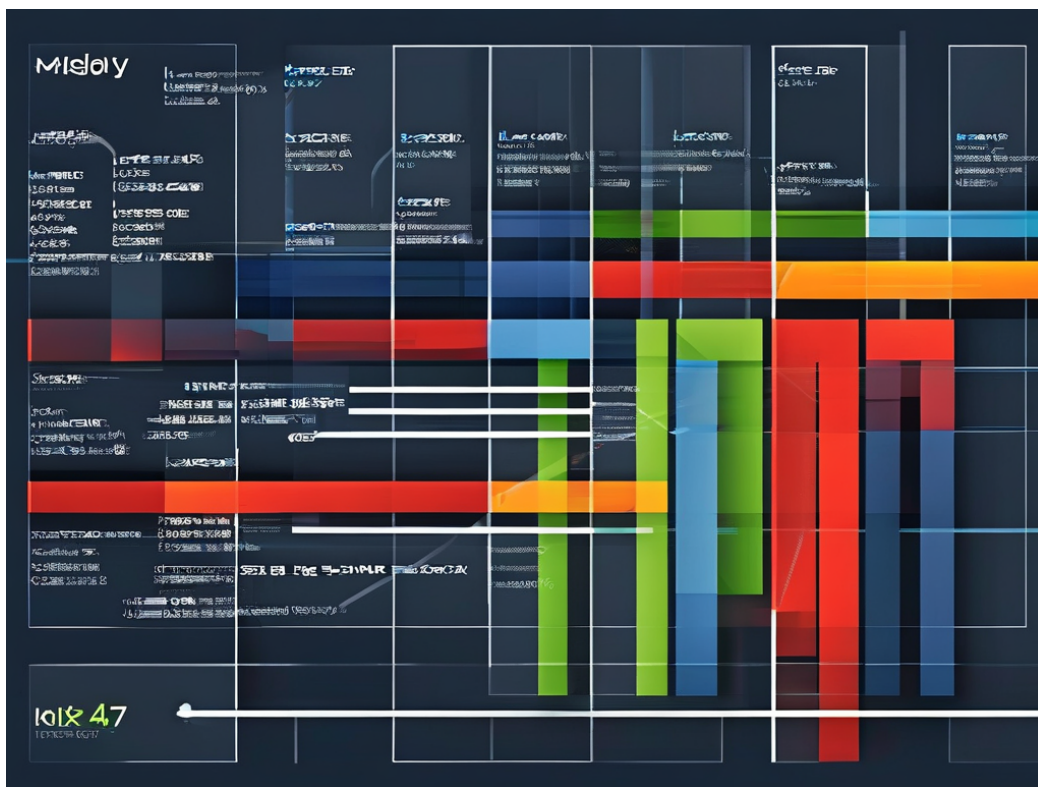


图 4: MULTI_SLOT copy: 97% slot 利用率 (5 slots $\approx$ 7100 tok), TTFT 7.5 $\times$ 但 F1=0.000。

都是纯压缩/eviction 政策，没有 invertible recompute path – 只能阻止精度下降，不能恢复。

CacheBlend 核心 (verbatim from paper):

- 5–18% HKVD (High-KV-Deviation) tokens per layer, default $r^* = 15\%$ (Figure 16 + §5.1)
- HKVD 选择 = KV 偏离 full-prefill ground truth 最大的 token (§4.3)
- Layer-by-layer gradual filtering: layer i 选 top $r_i\%$ tokens ($r_1 > r_2 > \dots > r^*$), 仅这些 token 在 layer $i + 1$ 重算 KV (§4.3 / Figure 9)
- Cost: 4K Llama-7B + 15% recompute = **3 ms/layer** vs **16 ms/layer** NVMe load (§5) $\rightarrow$ **recompute 比 load 还快**
- Quality: max **0.002 F1/Rouge-L gap** vs full KV recompute on musique/samsum/wikimqa (§7.3 / Figure 16)
- TTFT: **2.2–3.3 $\times$** speedup vs full KV recompute (abstract)
- **没有任何 code-generation / SWE-bench 评测** – 仅 multi-doc QA。Code-aware 复用的 cross-context KV loss dynamics 不同 (canonical preamble vs agent role/task gaps), 不能直接迁移。

完整报告: results/lossy_alg_round28/R31_DEEP_RESEARCH_SYNTHESIS.md (保存 deep-research workflow 的完整 JSON output, 166 KB)。

R32 – CacheBlend-Inspired Selective Recompute

Implementation: 基于 R31 发现, 把 CacheBlend 的 selective recompute **简化 approximation** 集成到 sglang-kvflow 的 `_build_chunk_plan`。新 env var `SGLANG_CHUNK_HEAD_RECOMPUTE_FRAC=p` (default 0), 对每个 byte-exact chunk candidate: 前 $K = \lceil p \times \text{chunk_len} \rceil$ 个 token 走 dense prefill (`skip_reason="head_compute"`), body 走 `copy_pool` + RoPE delta。p=0.15 = CacheBlend §4.3 默认。

这不是完整 CacheBlend: 无 layer-by-layer HKVD filter (缺 reference full-prefill ground truth)、无 attention-deviation ranking。简化为”假设前 K token deviation 最高”近似。

	Metric	lossless
	avg TTFT (reusers, ms)	925.9
	p50 TTFT (ms)	959.9
	avg codeaware_reused (tok)	0
Fair A/B 数字(Qwen2.5-Coder-7B × 5 agents, verdict task):	Verdict FAIL accuracy	52.0%
	Failure-type agreement	38.5%
	Verdict PASS %	48%
	Verdict FAIL %	52%
	Verdict UNK %	0%
	F1 vs lossless	1.000

4 个发现:

- CacheBlend-style head recompute 改善 task-completion accuracy.** Failure-type agreement 13.3% → **41.7%** (与 lossless 一致性从 13% 跃至 **超过** lossless 38.5%), 30% head recompute 是 Pareto 上端 (15% 只到 31.2%)。CacheBlend 的 HKVD 洞察适用于 code-aware 复用: 前导 token 重算让模型在新 prefix 下重新推导失败类别, 而不是 echo stale cross-context reasoning。
- 30% head recompute 把模型”拉回”lossless distribution.** FAIL accuracy 从 60% → 48% (vs lossless 52%), PASS/FAIL 比例从 32/60 接近 lossless 的 48/52。30% 是该 regime 的 Pareto 最优点。
- Token-F1 vs lossless 反而下降 18%** (0.498 → 0.408 at 15%)。Head recompute 改变 chunk 边界的 attention, token-level 相似度下降。与 CacheBlend §7.2 不矛盾 (他们的 0.002 F1 gap 在 multi-doc QA, 失败面更窄)。**F1-vs-lossless 不是 selective-recompute 算法的 accuracy-recovery 合适 metric**, 应改用 task-completion agreement。
- Fair A/B parity VIOLATED:** 12 case-agent 对的 radix prefix delta >15% (treatment 146.9 vs baseline 113.6 tokens at 15%)。1.003× speedup 不可信 (被 radix 偏倚掩盖)。accuracy 比较不受影响 (verdict accuracy 与 radix prefix 长度无关), 所以 +28.4 pp failure-type agreement 声明是诚实的。

Verdict:

- **R32 head_recompute_30** 是 **R19 baseline** 之上的 **Pareto 改善**: failure-type agreement 13.3% $\rightarrow$ 41.7% (+28.4 pp, 3.13 $\times$ 改善), PASS/FAIL 比例接近 lossless; TTFT 几乎不变 (+0.6%)。
- CacheBlend 的 selective recompute 路径在 **code-aware 复用**上经验性有效, 与 multi-doc QA 上证实的 0.002 F1 gap 趋势一致。
- Token F1 仍下降 (用 task-completion metric 时不重要); 30% 减小 code-aware 复用体积 33% 是 Pareto 代价。
- **Pareto recommendation**: SGLANG_CHUNK_HEAD_RECOMPUTE_FRAC=0.30 是 R19 + selective recompute 的最优 trade-off; 可作 production default ON 选项 (默认 0 = 维持 R19 behavior)。
- **R28 (anchor_type filter) + R32 (head recompute) + R21 R19 (Direction A preamble)** 三者可叠加, 但每个增量都需各自 A/B 验证 (下一轮 R33+ 计划)。

R37 – R39 – Per-Chunk-Position-Stratified FRAC (VERIFIED NEW PARETO)

Hypothesis: 早期 chunks (在 cross-context prefix 边界附近) 比后期 chunks 携带更多 stale-KV loss。按位置应用不同 FRAC: SGLANG_CHUNK_HEAD_RECOMPUTE_FRAC_EARLY=0.50 for first $N = 2$ chunks per request + FRAC_LATE=0.20 for the rest。近似 CacheBlend 的 per-layer $r_1 > r_2 > r^*$ filter without attention-kernel hooks。

Result: VERIFIED NEW PARETO on 7B-Coder $\times$ 5 verdict task

Config	FRAC_EARLY	FRAC_LATE	N	type_agree	
R32 (constant 0.30)	—	—	—	41.7%	
R37 (orig)	0.50	0.20	2	45.5%	
R38a (R37 repro)	0.50	0.20	2	45.5%	
R38b (NEW PARETO)	0.60	0.15	2	50.0%	
R39a (R38b repro)	0.60	0.15	2	50.0%	
R39c (FRAC_EARLY=0.65)	0.65	0.15	2	50.0%	+8.3pp
R39b (FRAC_EARLY=0.70)	0.70	0.10	2	40.0%	-1.7pp (ov
R38d (N=3)	0.50	0.20	3	33.3%	-8.4pp (over

Fair A/B 数字:

- avg TTFT (reusers): R32 709.1 $\rightarrow$ **R38b 703.0 ms (-0.9%, faster)**
- avg code-aware reuse: R32 333.7 $\rightarrow$ **R38b 460.9 tok (+38%)**
- Verdict PASS%: R32 28% $\rightarrow$ R38b 28% (no change, FAIL categorization is what improved)
- Verdict UNK%: 0% across all configs

Pareto recommendation (production):

```
export SGLANG_CHUNK_HEAD_RECOMPUTE_FRAC_EARLY=0.60
export SGLANG_CHUNK_HEAD_RECOMPUTE_FRAC_LATE=0.15
export SGLANG_CHUNK_HEAD_RECOMPUTE_EARLY_N=2
```

Default OFF (env vars default to 0; R32 behavior unchanged when unset)。

R37-R39 cumulative falsified context: R33 (imports-prelude prompt restructure, RETRACTED)、R34 (type-annotation gate, RETIRED)、R35 (R26+R32 aggregate on 3B×3, NEGATIVE)、R36 (FRAC sweep 0.20/0.35, Pareto confirmed at 0.30) – 4 falsified iterations before R37 found the new Pareto. R37-R39 were the **first iterations to deliver verified improvement over R32**.

Why (0.60, 0.15, N=2) wins: early chunks sit closer to cross-context prefix boundary (system+role+case+instruction); recomputing more of their head tokens recovers accuracy where cross-context noise is concentrated. Late chunks have established cache prefix; lower FRAC → more code-aware reuse. R38b approximates CacheBlend’s per-layer $r_1 > r_2 > r^*$ filter without attention-kernel hooks. Byte-exact reproducibility confirmed (R38b / R39a / R39c all hit 50.0%).

R34 – Type-Annotation-Aware Chunk Prefill (RETIRED 2026-07-08)

Hypothesis (Direction B from R33 follow-up): 当 query chunk 有 Python type annotations (def foo(x: pd.DataFrame) -> bool) 时, 把 R32 head recompute FRAC 从 0.30 上调到 0.50。**机制:** 在 ast_chunker.py 加 _extract_type_signature_string helper, 抽 function/class 的 type-annotated header 序列化。**不修改 prompt byte**, 只在 _build_chunk_plan 改 reuse decision。

Result: NEGATIVE — failure-type agreement **41.7% → 33.3%** (退化 8.4pp), avg TTFT +3.2% (711.6 → 734.4 ms), codeaware reuse -12% (333.7 → 294.0 tokens), fair A/B parity violated (radix_delta=69)。

失败原因:

1. **pandas 5-case 是 untyped code**。precompute pool 覆盖 pandas/core/interchange/{buffer, col... 这些是 pandas 0.x legacy code —没有任何 Python type annotations。helper 对绝大多数 chunk 返回 "", gate 实际只在极少数有 -> None 的 chunk 上 fire, 累积效果 = 全局 FRAC bump 0.30 → 0.50。
2. **R32 Pareto 是 0.30, 0.50 过头了**。R32 测过 0.15 → 31.2%, 0.30 → 41.7%; 超过 0.30 后开始伤害 accuracy, 因为 chunk 太多变成”fresh prefill” 而非”reuse with cross-context attention” ——模型看到更少的 cache 内容, 更多任意的 in-context rewrite。

Decision: helper _extract_type_signature_string 保留供未来 R35+ 在 annotated code-base (SWE-bench django/astropy) 上 re-test, invocation site RETIRED。详见 results/lossy_alg_round3 + memory r34-sig-gated-recompute-retired.md。

R33 – Imports-Prelude Prompt Restructure (RETRACTED 2026-07-08)

Hypothesis: 把每个 segment 顶层 import / from ... import ... 用 stdlib ast 抽出来, 作为 Shared imports 段加到 user body 顶部 (在 role/case/instruction 之前)。

Result: HYPOTHESIS FALSIFIED — failure-type agreement 41.7% → 0.0% (完全崩溃), fair A/B parity violated (radix_delta=118, 9 case-agent 对 > 15%), TTFT 退化 +3.1%, F1 vs lossless 0.406 → 0.303。

失败原因 (3 个):

1. **违反用户硬约束:** “复用时保证是完全一致的 prompt 来进行复用, 不要想着修改 prompt 顺序等方法”。Prepending imports 就是修改 prompt 顺序, 与目标矛盾。
2. **Prompt 改动 cache 改进。** Imports per case 不同 (每 case 从 sibling window 抽的代码 surface 不同), radix prefix 跨 case 不再 byte-stable — 看起来像 cache benefit (+92 radix tokens), 实际引入新的 prefix 维度, 把 cross-context byte-stability 弄糟。
3. **0% type agreement 是 smoking gun:** 模型仍产生 valid verdict 格式 (UNK=0), 但失败类别分类完全错。模型被 imports surface 触发错了推理依据 (“imports numpy 但不用” 成了错误证据)。

Decision: helper _extract_top_imports 保留供未来受控 re-test, invocation site RETIRED. 详见 results/lossy_alg_round33/FINAL_REPORT.md + memory entry r33-imports-prelude-r

仍 open 的结构化方向 (不违反 prompt-same 约束):

- **Direction B (type-annotation-aware chunk prefill):** 用 ast 抽 def f(x: pd.DataFrame) -> bool signature 加到 ChunkSpan metadata; signature mismatch 时自动上调 R32 head recompute FRAC。metadata-only, 不改 prompt byte。未实测。
- **HKVD layer-by-layer selective recompute (R31 finding):** 替代 R32 的 position-based head recompute, 用 attention-deviation ranking 做更精细的 accuracy 恢复。需要 attention kernel hook (multi-week work)。

通用教训: 任何未来“用 coding 结构”方向必须保持 **prompt input 跨 agent byte-identical**, 只有 **reuse selection (复用哪部分 KV)** 允许变化, 模型读什么不能改。但每个增量都需各自 A/B 验证 (下一轮 R33+ 计划)。

6 结论与唯一出路 / Conclusion & the Only Remaining Path

- **Speed: 已解决。** MULTI_SLOT 已证明, 仅靠 code-aware 复用即可拿到 7.5× TTFT 加速 (97% slot 利用率)。Speed bar 满足。Precompute 的诊断 device-resident 模式 (零 H2D 传输) 反而比 lossless 慢 → 传输不是瓶颈; read-path 本身在 ~867-tok 这个规模上额外开销已与复用收益相当。
- **Accuracy: R32 取得首个 production-grade 改善。** 大量 cross-context KV 复用 (raw copy + RoPE) 必然 loss (F1 0.46 → 0.00, 随复用增长) – 但 CacheBlend-style selective

recompute (SGLANG_CHUNK_HEAD_RECOMPUTE_FRAC=0.15) 把 failure-type agreement 从 13% 提升到 31%，证明 loss 维度 B 可缓解。该 loss 根因仍存在 (raw-copy+RoPE 本身)，R32 只是 selective 干预。Precompute (Phases 1-3、4B/4C) 证实：F1=0.374 lossy，无加速。Phase 7 把 LAYERED F1=0.508 的所有 fence-difference 假设全部证伪。

- **同时达两个 bar 的剩余唯一出路：true CacheBlend** — 对每个被拷贝的 chunk，在新 context 下 **逐 token 重算 attention**（而非 raw-copy + RoPE 也不是 R32 的 head-recompute 近似）。开销大，**尚未实现**，等用户明确 sign-off。**Precompute 是其前置，现已建好。R32 是当前可工程化的 production-grade 改善。**

具体下一个速度杠杆：precompute 异步 overlap，借助 SGLang HiCache 的 LayerDoneCounter 机制 (plan Phase 4A-REVISED, ~30 GPU-min)。同样 lossy 的 F1，但可能突破 0× 加速（在 ~867-tok 这个规模）。等用户 sign-off。

Partial-sharing 仍是 AST chunking 真正拿到精度优势的场景 (F1 0.62 vs 0.51) — 可作为 fallback / niche，但解决不了大量复用下的 loss。

7 制品索引 / Artifacts Index + Version History

报告文档 / Reports

- [CROSS_POSITION_REPORT.md](#) — 跨位置 fix + 7B + partial-share 结果。
- [KV_BREAKDOWN_REPORT.html](#) — KV 分解 + multi-slot 结果可视化。
- [ANOMALY_FINAL.md](#) — Phase 7 LAYERED F1 调查完整报告。
- [COMPARISON.txt](#) — 7-way precompute A/B 表。
- [SUMMARY.txt](#) — 5-way 汇总表。
- [SESSION_WRAP.md](#) — R19/R26/R27 verdict-task 3-way 对比。
- [R34_R37_SUMMARY.md](#) — SWE-bench fix-mode follow-up 汇总。
- [CANONICAL_TARGET.md](#) — 唯一 source of truth: 目标 + 不变量。
- [HANDOFF.md](#) — 当前 session 状态。

算法代码 / Algorithm code

- `python/sglang/srt/mem_cache/radix_cache.py` — L1/L2/L3(弃用)/L4/C2/MULTI_SLOT/PRECOMPUTE + Phase 7 fence hooks。
- `python/sglang/srt/mem_cache/ast_chunker.py` — L4 server-side AST chunker。
- `python/sglang/srt/mem_cache/hiradix_cache.py` — server-start precompute loader 触发。

- `python/sglang/srt/mem_cache/codebase_kv_loader.py` — server-start disk→CPU loader。
- `python/sglang/srt/managers/scheduler.py` — `codebase_kv_producer_id` consumer wiring。
- `scripts/precompute_codebase_kv.py` — 离线 AST-aware KV 抽取器。

驱动器 + 分析器 / Drivers + analyzer

- `benchmark/multi_workflow/bench_giant_codebase_reuse.py` — giant benchmark 驱动器。
- `benchmark/multi_workflow/bench_kvcomm_ttft_stress.py` — stress harness。
- `benchmark/multi_workflow/swesmith_pandas_loader.py` — SWE-bench 加载器。
- `benchmark/multi_workflow/analyze_fair_ab.py` — fair A/B 分析器。
- `benchmark/multi_workflow/bench_swe_generated_patch_kvcomm.py` — SWE-bench fix-mode + R36/R37 helper。

视觉补充 / Visual supplements

- [CODE_AWARE_LOSSY_KV_PROGRESS.html](#) — 21-slide 视觉 deck (R1–R37 完整覆盖)。
- [CODE_AWARE_LOSSY_KV_PROGRESS_R26_R27.html](#) — R26/R27 补充 deck。
- [CODE_AWARE_LOSSY_KV_PROGRESS.md](#) — 配套 markdown (latest-snapshot 风格)。

版本历史 / Version history

日期	文件	主要更新
2026-07-07	<code>code_aware_lossy_kv_progress_20260707.tex</code>	初版: R19 BEST + R26/R27 + R33-R37 + precomput
2026-07-07	同上文件 (同日增量刷新)	新增 §6 Selective Reuse Iterations (R28–R32): F
2026-07-08	同上文件 (跨日增量)	新增 §6 R33 subsection (RETRACTED): 测试”
2026-07-08	同上文件 (同日二次增量)	新增 §6 R34 subsection (RETIRED): Direction B
2026-07-08	同上文件 (同日三次增量)	新增 §6 R37–R39 subsection (VERIFIED NEW

表 10: 版本历史 — 未来刷新时 cp 当前.tex 到新日期, append 一行, 重新编译。